Software Testing Project

TOPIC: “SMART LIBRARY CIRCULATION AND POLICY MANAGEMENT SYSTEM”



AUTHORS:

- VIOLA ZENELI
- SIDRIT RRUSTEMI
- MEGI ALMADHI
- ARSILDO VELIU

Project Overview

Complete Testing Process for Smart Library Management System

This project presents a comprehensive overview of the **complete testing process** of a Smart Library Circulation and Policy Management System. The system was fully implemented before testing began, ensuring that all testing activities applied to a real, working software product, highlighting the importance of structured testing methodologies.



System Design

Clean and Modular Architecture for Optimal Performance and Maintenance

The architecture of the Smart Library Circulation and Policy Management System was carefully crafted to enhance **maintainability** and **testability**. By adopting a modular design, each component can be developed, tested, and maintained independently, allowing for easier updates and improved collaboration among development teams as needs evolve.

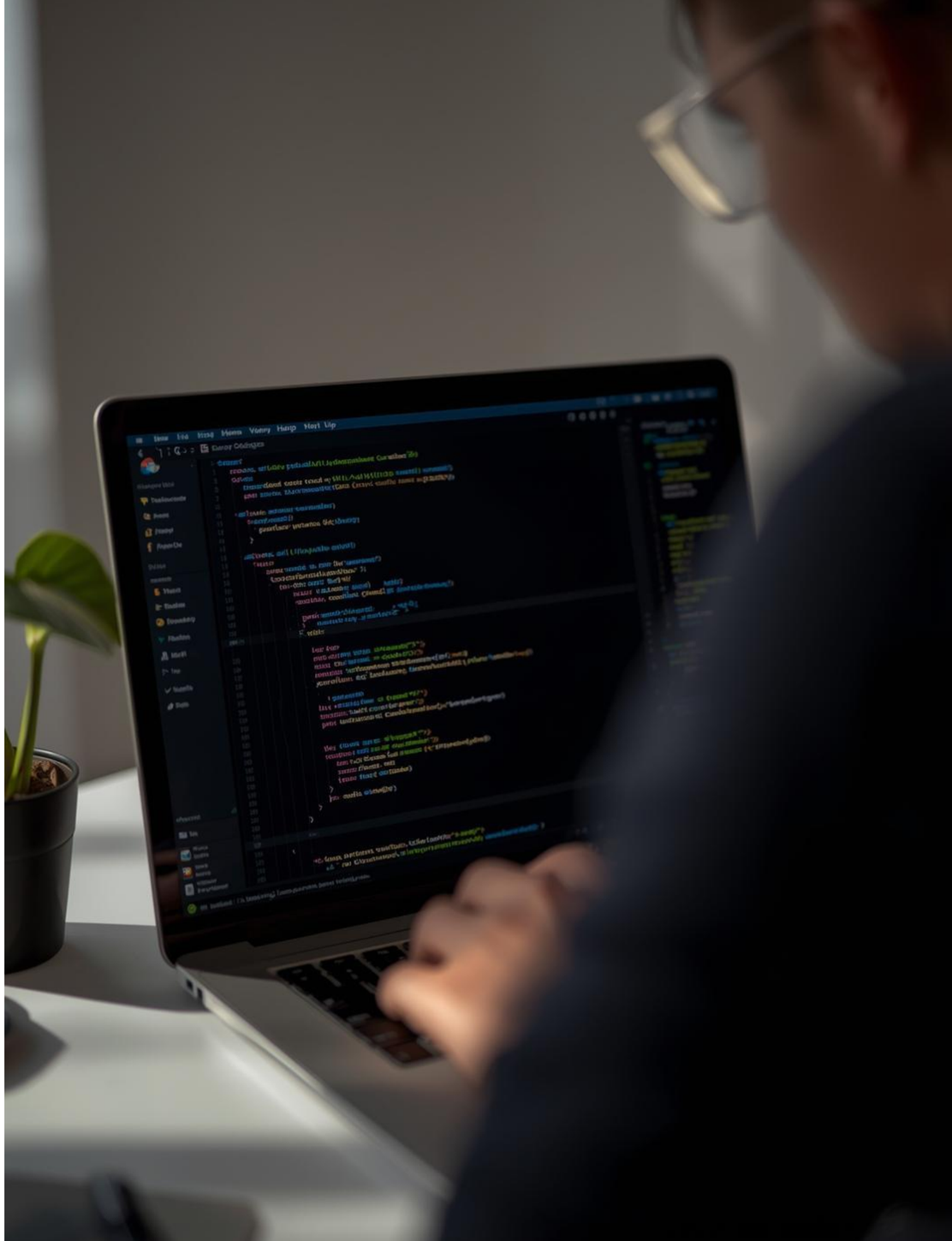


Static Testing

An essential part of the software testing process

Static testing is crucial for identifying issues during the early stages of development. This process involved thorough code reviews and documentation checks, ensuring that potential defects were addressed before executing the system. By eliminating errors upfront, we improved overall software reliability and minimized later-stage testing efforts.

- ***Arsildo Veliu:*** *DefaultLibraryFacade, BorrowRecord, LibraryItem (3 classes)*
- ***Megi Almadhi:*** *Member, StubAlertService, DefaultBorrowPolicy (3 classes)*
- ***Viola Zeneli:*** *DefaultPenaltyPolicy, DefaultCirculationService, MemoryBorrowStorage (3 classes)*
- ***Sidrit Rrustemi:*** *MemoryItemStorage, MemoryMemberStorage, all interfaces and enumerations (3+ components)*



Testing Analysis



Boundary Value Testing (BVT)

Viola Zeneli:

- Method: DefaultCirculationService.calculateLateFeeWithDiscount(String itemCode, int discountPercent)
- Type: Robust BVT
- Parameter tested: discountPercent with boundaries [0, 100]
- Test cases: 7 (covering -1, 0, 1, 50, 99, 100, 101)

Sidrit Rrustemi:

- Method: DefaultCirculationService.shouldSendReminder(String itemCode, int daysBeforeDue)
- Type: Robust BVT
- Parameters tested: daysBeforeDue [0, 7] and derived daysUntilDue
- Test cases: 9 (covering -1, 0, 1, 3, 6, 7, 8 and overdue scenarios)

Megi Almadhi:

- Method: DefaultCirculationService.getRemainingBorrowCapacity(int memberId)
- Type: Robust BVT
- Boundary tested: Maximum loans (N = 5)
- Test cases: 3 (testing N-1, N, N+1 scenarios)
- Formula validated: Math.max(0, maxLoans - activeBorrows)

Arsildo Veliu:

- Method: DefaultCirculationService.getDaysUntilDue(String itemCode)
- Type: Robust BVT
- Boundary tested: Due date equals current date
- Test cases: 3 core boundary cases (yesterday: -1, today: 0, tomorrow: +1) plus edge cases (null code, empty code, no borrow, returned item)



Equivalence Class Testing

Viola Zeneli:

- Method: DefaultCirculationService.assessItemReturnStatus(String itemCode, int daysLate)
- Type: Weak Robust Equivalence Class Testing
- Classes identified: 9 (invalid itemCode, no active borrow, already returned, early return, on-time, slightly late, moderately late, very late, severely overdue)
- Test cases: 9

Sidrit Rrustemi:

- Method: DefaultCirculationService.validateMemberStatus(int memberId)
- Type: Weak Robust Equivalence Class Testing
- Classes identified: 10 (invalid ID, member not found, multiple suspension states, single issues, good standing)
- Test cases: 10

Megi Almadhi:

- Method: DefaultCirculationService.getFineCategory(String itemCode)
- Type: Weak Robust Equivalence Class Testing
- Classes identified: 7 (invalid input, no borrow, returned, on-time, low fine, medium fine, high fine)
- Test cases: Multiple representative values per class (minimum, typical, maximum)

Arsildo Veliu:

- Method: DefaultCirculationService.getMembershipEligibility(int memberId, int currentBorrows, boolean hasOverdue, boolean isActive)
- Type: Weak Robust Equivalence Class Testing
- Classes identified: 7 (invalid borrow count, inactive membership, overdue items, excellent/good/moderate/heavy usage standing)
- Test cases: Multiple representative and boundary values per class



Code Coverage Testing

Viola Zeneli:

- Code Coverage Testing
- Method: DefaultCirculationService.canRenewBorrow(int memberId, String itemCode)
- Coverage types:
  - Statement Coverage: 3 test cases
  - Branch Coverage: 3 test cases
  - Condition Coverage: 12 test cases (11 boolean conditions identified)
  - MC/DC Coverage: 3 decisions tested with 12 total test cases

Sidrit Rrustemi:

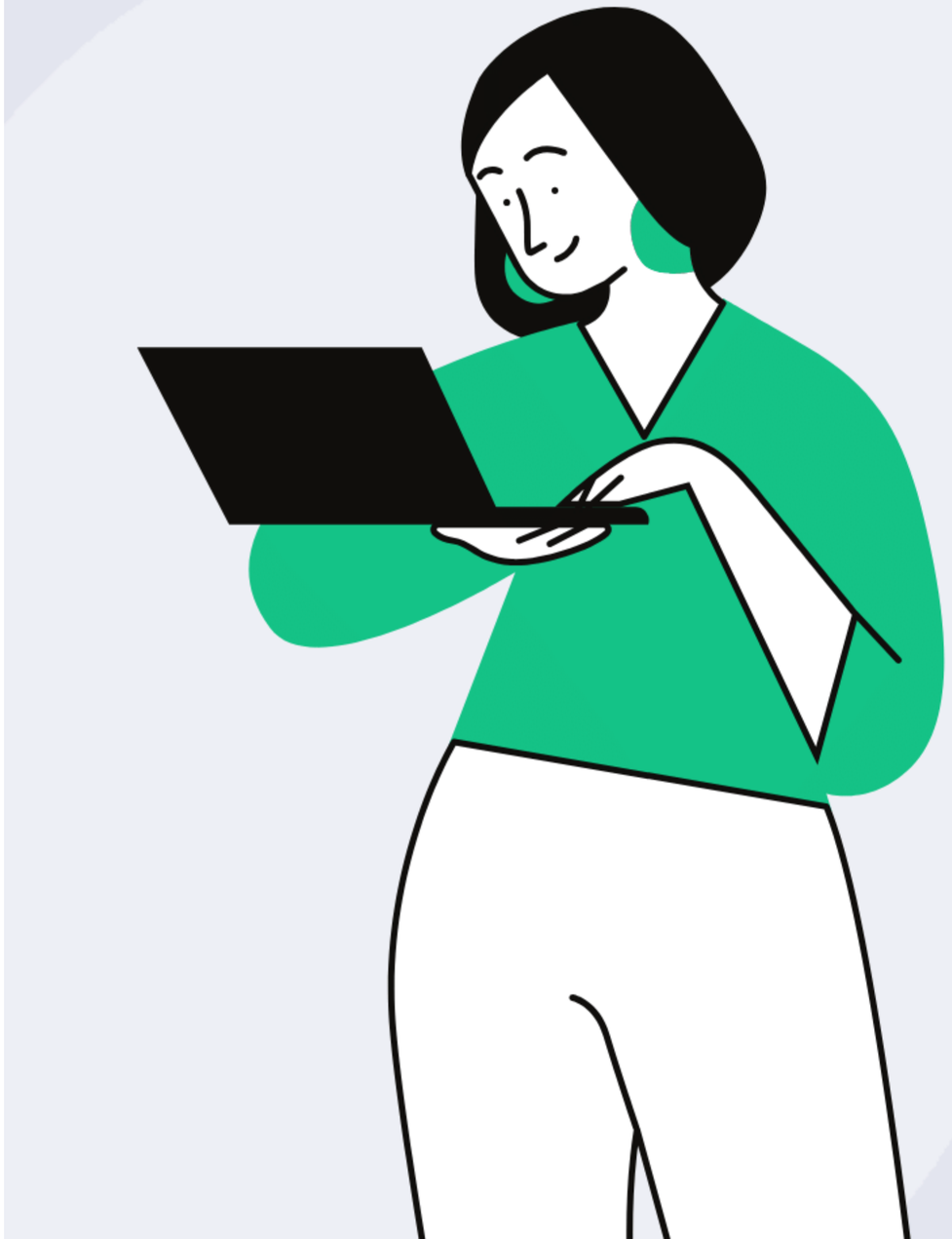
- Method: DefaultBorrowPolicy.canBorrow(Member member, int activeBorrows, boolean hasOverdueBooks)
- Coverage types:
  - Statement Coverage: 1 test case
  - Branch Coverage: 2 test cases
  - Condition Coverage: 8 test cases (4 boolean conditions identified)
  - MC/DC Coverage: 5 test cases

Megi Almadhi:

- Method: DefaultCirculationService.shouldSendReminder(String itemCode, int daysBeforeDue)
- Coverage types:
  - Statement Coverage: 100% - all executable lines tested
  - Branch Coverage: 100% - all decision outcomes tested
  - Condition Coverage: 100% - all atomic conditions tested independently
  - MC/DC Coverage: 100% - each condition independently affects outcome

Arsildo Veliu:

- Method: DefaultCirculationService.canBorrowItemType(int memberId, String itemType, double itemValue)
- Coverage types:
  - Statement Coverage: Complete coverage with valid borrowing scenario
  - Branch Coverage: All validation branches tested (invalid ID, null type, negative value, member not found)
  - Condition Coverage: 4 conditions tested (isActive && hasCapacity && noOverdue && valueLimitOk)
  - MC/DC Coverage: 5 test cases (base + 4 condition variations)
- Additional tests: Membership type value limits (Student: $100, Staff: $500, Guest: $50)



Unit Testing

Ensuring accuracy through isolated class verification and dependency management

In our project, **unit tests were crucial** for verifying individual classes using Mockito. This approach allowed us to isolate dependencies, ensuring that each component functioned correctly in isolation before integration. By focusing on these individual units, we could confidently build a robust and reliable system.

Arsildo Veliu:

1. DefaultLibraryUnitTest (6 test cases)
2. StubAlertServiceUnitTest (1 test case)
3. MemoryBorrowStorageUnitTest (12 test cases)
4. MemoryItemStorageUnitTest (5 test cases)

Megi Almadhi:

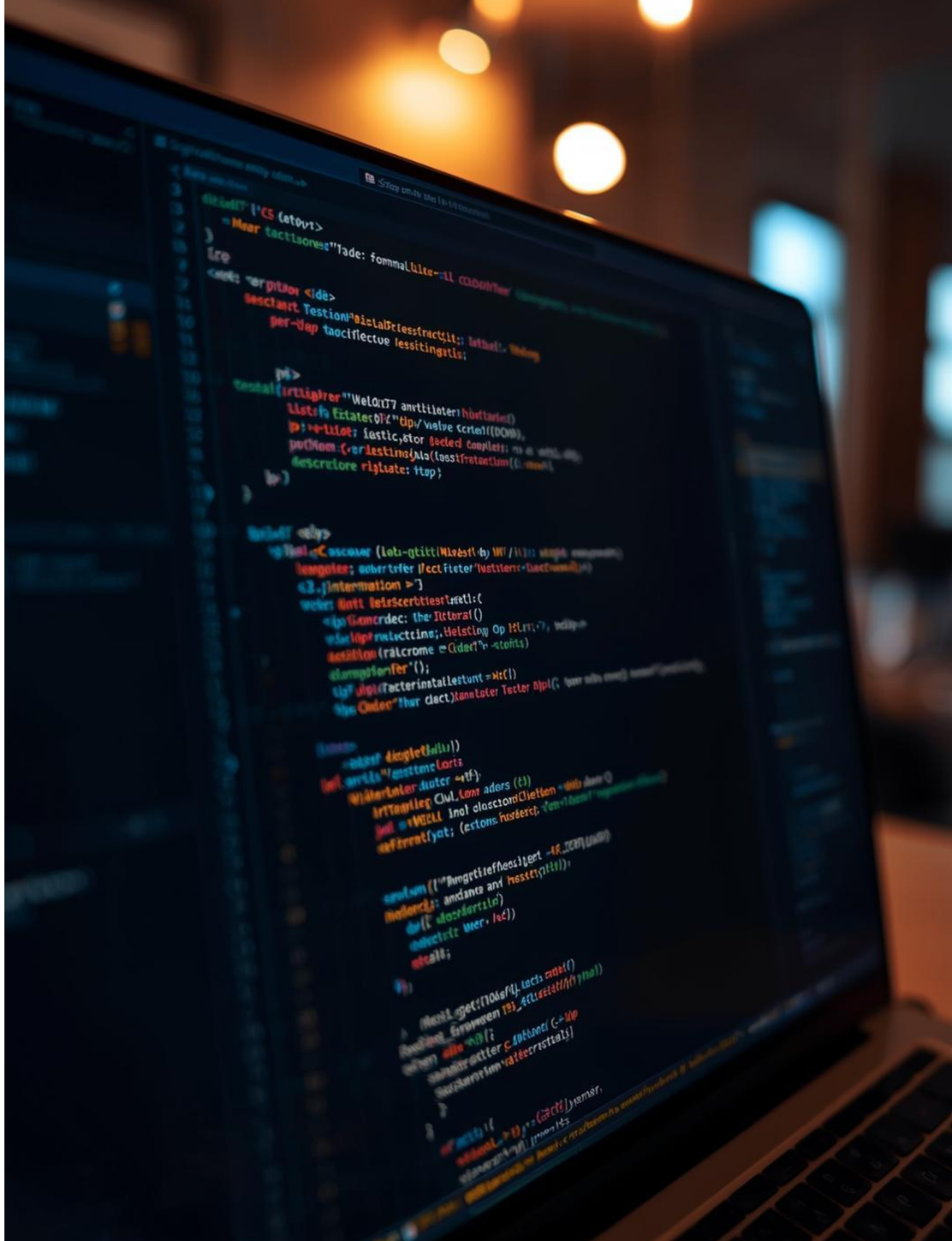
1. BorrowRecordUnitTest (12 test cases)
2. LibraryItemUnitTest (2 test cases)
3. MembershipTypeUnitTest (1 test case)
4. MemberUnitTest (1 test case)
5. DefaultBorrowPolicyUnitTest (5 test cases)
6. DefaultPenaltyPolicyUnitTest (1 test case)
7. MemoryMemberStorageUnitTest (3 test cases)

Viola Zeneli:

1. DefaultCirculationServiceUnitTest (36 test cases - first half)

Sidrit Rrustemi:

1. DefaultCirculationServiceUnitTest (38 test cases - second half)



Integration Testing

Validating Real Components for Seamless Functionality

Integration testing was crucial in ensuring that all components of the Smart Library Circulation System function together as intended. By utilizing in-memory storage, we validated the interactions between various real components, ensuring data consistency and performance under simulated conditions, which are essential for a fully operational software product.

Integration testing followed a **bottom-up approach**, starting with the lowest-level components (A) and progressively integrating higher-level modules until the complete system functionality was validated.

- **Megi Almadhi:**
  A_MemoryStoragesIntegrationTest (8 test cases)
  B_PolicyIntegrationTest (8 test cases)
- **Viola Zeneli:**
  C_CirculationServiceIntegrationTest (16 test cases - first half)
- **Sidrit Rrustemi:**
  C_CirculationServiceIntegrationTest (16 test cases - second half)
- **Arsildo Veliu:**
  D_LibraryFacadeIntegrationTest (12 test cases)



System Testing

Validating User Workflows Without Mocks in Real Systems

System testing was an essential phase where the entire user workflows were validated without any mocks. This approach ensured that all features functioned together seamlessly in a live environment, providing confidence in the system's performance and reliability prior to deployment. Such validation is crucial for a successful user experience.

Arsildo Veliu:

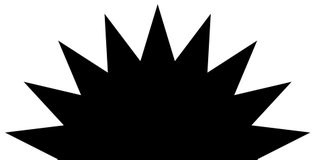
System test cases: 4

- borrowFlow_success_marksItemUnavailable_andCreatesActiveBorrowRecord()
- returnFlow_success_marksRecordReturned_andMarksItemAvailable()
- borrowFlow_nonexistentMember_throwsRuntimeExceptionWithMessage()
- borrowFlow_nonexistentItem_throwsRuntimeExceptionWithMessage()

Megi Almadhi:

System test cases: 4

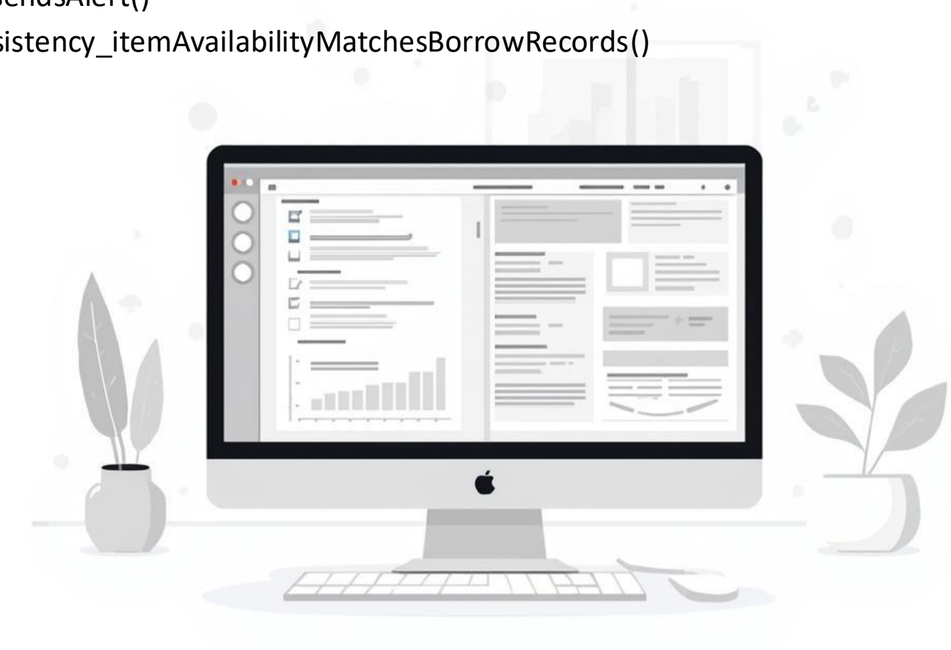
- completeUserJourney_borrowMultipleItems_returnSome_borrowMore()
- studentMember_canBorrow3Items_cannotExceed()
- borrowUpToLimit_returnOne_borrowAnother_respectsCapacity()
- borrowReturn_sameItem_multipleTimes()



Viola Zeneli:

System test cases: 4

- twoMembers_borrowAndReturn_differentItems_noInterference()
- memberWithOverdueItem_cannotBorrowAnother()
- returnOverdueItem_sendsAlert()
- systemMaintainsConsistency_itemAvailabilityMatchesBorrowRecords()



Sidrit Rrustemi:

System test cases: 4

- staffMember_canBorrow5Items_cannotExceed()
- guestMember_limitedTo1Item()
- twoMembers_cannotBorrowSameItem_secondMemberBlocked()
- borrowFlow_inactiveMember_throwsRuntimeException()

Test Documentation

Comprehensive information on test cases and their management

Test documentation is integral to the software testing process. Each test case was crafted to include essential details such as the **description**, **purpose**, **expected result**, **actual result**, **test type**, and the responsible team member. This structured format ensures clarity and traceability throughout the testing lifecycle.



Tools and Technologies

Key technologies utilized for the software testing project

For this project, we employed a robust stack of technologies including Java for programming, Maven for project management, and JUnit 5 for testing. Mockito was used for unit testing, while IntelliJ IDEA served as our integrated development environment. GitHub facilitated version control and collaboration throughout the development process.



Project Outcomes

Gaining Practical Insights into Software Testing Practices and Processes



The project provided **hands-on experience** with real-world software testing, allowing the team to engage with various testing methodologies effectively. Participants gained valuable insights into the complexities of software testing, enhancing their understanding of both theoretical concepts and practical applications in a real system environment.

Conclusion

A Comprehensive Look at Software Testing Lifecycles

This project successfully showcased the **complete software testing lifecycle** applied to a real-world system. By focusing on structured testing strategies and clear documentation, the team effectively ensured product quality and functionality. The hands-on experience gained throughout the process illustrates the significance of thorough testing in software development.

